1. Data Loading And Initial check

In [5]:
```
import pandas as pd

# Load the dataset
df = pd.read_csv('data.csv')

# 1. Basic shape
print("Shape:", df.shape)

# 2. Column names + data types
print("\nData Types:\n", df.dtypes)

# 3. First 5 rows
print("\nFirst 5 rows:\n", df.head())

# 4. Missing values
print("\nMissing values per column:\n", df.isnull().sum())

# 5. Basic stats for numeric columns
print("\nDescriptive stats:\n", df.describe())
```

```
Shape: (1200, 14)

Data Types:
 OrderID               str
Date                   str
CustomerID             str
Product                str
Quantity             int64
UnitPrice          float64
ShippingAddress        str
PaymentMethod          str
OrderStatus            str
TrackingNumber         str
ItemsInCart          int64
CouponCode             str
ReferralSource         str
TotalPrice         float64
dtype: object

First 5 rows:
      OrderID        Date CustomerID  Product  Quantity  UnitPrice  \
0  ORD200000  2023-01-04     C72649  Monitor         5     570.62
1  ORD200001  2024-08-23     C75739    Phone         2     151.35
2  ORD200002  2024-02-27     C81728   Tablet         5     550.68
3  ORD200003  2023-10-15     C33540    Chair         1     273.19
4  ORD200004  2025-05-08     C81840  Printer         4     626.01

  ShippingAddress PaymentMethod OrderStatus TrackingNumber  ItemsInCart  \
0     928 Main St    Debit Card     Shipped    TRK37947903            7
1     823 Main St        Online     Shipped    TRK91186779            3
2     512 Main St   Credit Card   Cancelled    TRK42903982            8
3     275 Main St    Debit Card    Returned    TRK62788070            5
4     668 Main St        Online   Delivered    TRK29241424            8

  CouponCode ReferralSource  TotalPrice
0     SAVE10      Instagram     2853.10
1     SAVE10       Referral      302.70
2   FREESHIP          Email     2753.40
3     SAVE10       Facebook      273.19
4     SAVE10          Email     2504.04

Missing values per column:
 OrderID              0
Date                  0
CustomerID            0
Product               0
Quantity              0
UnitPrice             0
ShippingAddress       0
PaymentMethod         0
OrderStatus           0
TrackingNumber        0
ItemsInCart           0
CouponCode          309
ReferralSource        0
TotalPrice            0
```

```
dtype: int64

Descriptive stats:
           Quantity    UnitPrice  ItemsInCart   TotalPrice
count  1200.000000  1200.000000  1200.000000  1200.000000
mean      2.945833   356.412750     5.485000  1053.968300
std       1.407557   197.177146     2.281983   819.856558
min       1.000000    11.390000     1.000000    11.390000
25%       2.000000   186.062500     4.000000   410.520000
50%       3.000000   364.210000     5.000000   823.615000
75%       4.000000   521.570000     7.000000  1578.475000
max       5.000000   699.930000    10.000000  3456.400000
```

2. Outlier Detection - IQR Vs Z-Score

In [6]:

```python
import numpy as np
from scipy import stats

# Function for IQR outliers
def find_iqr_outliers(col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    return outliers, lower, upper

# 1. IQR Method for TotalPrice
tp_outliers, tp_low, tp_high = find_iqr_outliers('TotalPrice')
print(f"TotalPrice IQR Outliers: {len(tp_outliers)} rows")
print(f"Normal range: {tp_low:.2f} to {tp_high:.2f}")

# 2. Z-Score Method for TotalPrice
df['TotalPrice_zscore'] = np.abs(stats.zscore(df['TotalPrice']))
z_outliers = df[df['TotalPrice_zscore'] > 3]
print(f"\nTotalPrice Z-Score Outliers: {len(z_outliers)} rows")

# 3. IQR for UnitPrice
up_outliers, up_low, up_high = find_iqr_outliers('UnitPrice')
print(f"\nUnitPrice IQR Outliers: {len(up_outliers)} rows")
print(f"Normal range: {up_low:.2f} to {up_high:.2f}")

# Show 3 sample outlier rows
print("\nSample TotalPrice Outliers:\n", tp_outliers[['OrderID', 'Product', 'TotalP
```

```
TotalPrice IQR Outliers: 8 rows
Normal range: -1341.41 to 3330.41

TotalPrice Z-Score Outliers: 0 rows

UnitPrice IQR Outliers: 0 rows
Normal range: -317.20 to 1024.83

Sample TotalPrice Outliers:
        OrderID  Product  TotalPrice
107  ORD200107  Printer     3353.75
326  ORD200326   Laptop     3352.40
328  ORD200328   Tablet     3370.20
```

3. Visual Analysis - Boxplot & Histogram

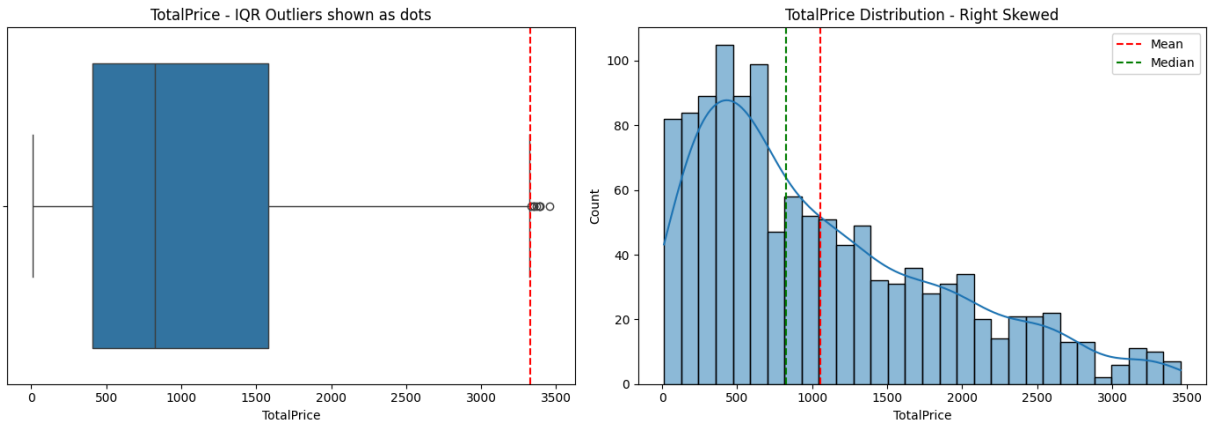
In [7]:
```
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Boxplot shows the IQR fence visually
sns.boxplot(x=df['TotalPrice'], ax=axes[0])
axes[0].set_title('TotalPrice - IQR Outliers shown as dots')
axes[0].axvline(3330.41, color='red', linestyle='--', label='Upper IQR Fence')

# 2. Histogram shows the skew
sns.histplot(df['TotalPrice'], bins=30, kde=True, ax=axes[1])
axes[1].set_title('TotalPrice Distribution - Right Skewed')
axes[1].axvline(df['TotalPrice'].mean(), color='red', linestyle='--', label='Mean')
axes[1].axvline(df['TotalPrice'].median(), color='green', linestyle='--', label='Me

plt.legend()
plt.tight_layout()
plt.show()
```



4. Correlation & Business Insights

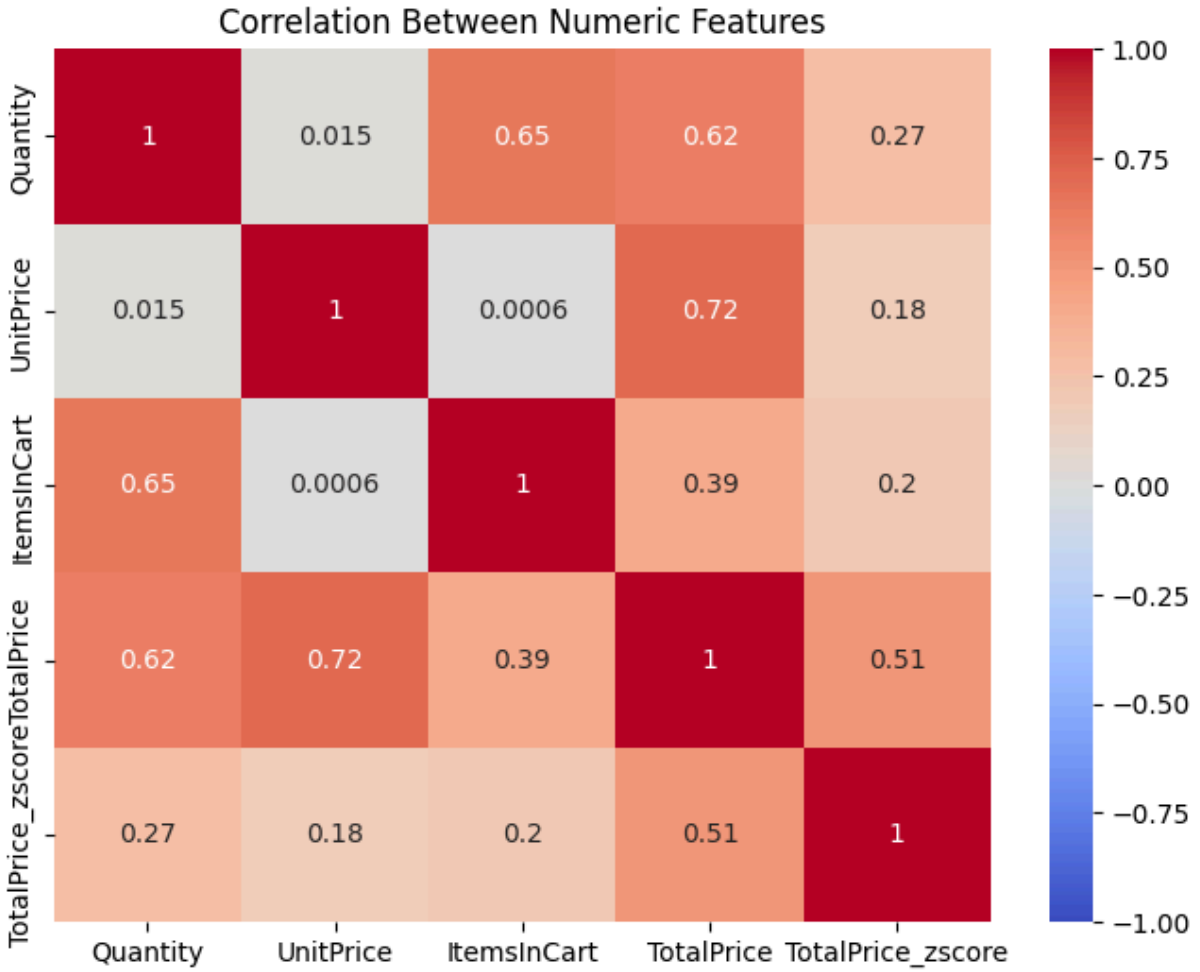
In [8]:
```
# 1. Correlation heatmap - safe version
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
plt.figure(figsize=(8,6))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Between Numeric Features')
plt.show()

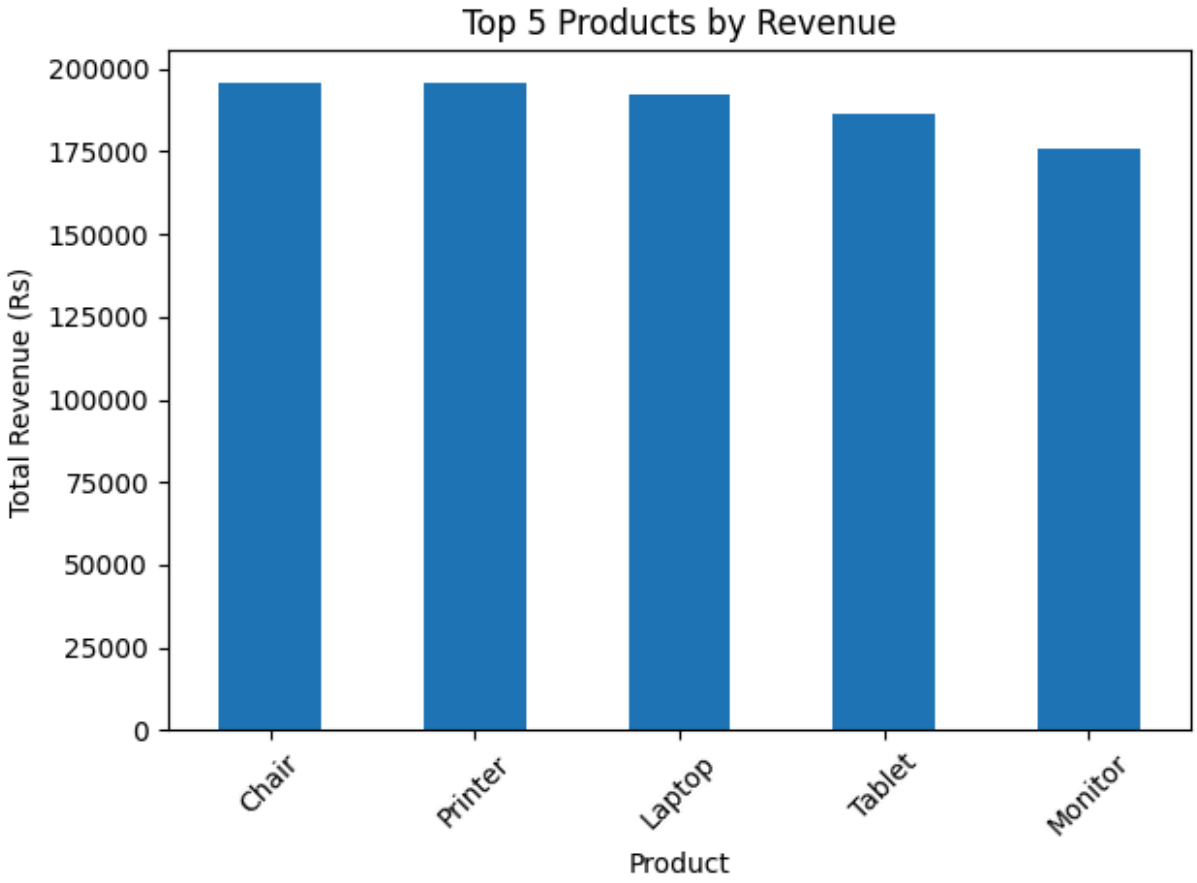
# 2. Top products by revenue
top_products = df.groupby('Product')['TotalPrice'].sum().sort_values(ascending=Fals
print("Top 5 Products by Revenue:\n", top_products)

top_products.plot(kind='bar', title='Top 5 Products by Revenue')
plt.ylabel('Total Revenue (Rs)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 3. Revenue by category - ONLY run if the column exists
if 'ProductCategory' in df.columns: # <-- change this to your column name
    top_categories = df.groupby('ProductCategory')['TotalPrice'].sum().sort_values(
    print("\nRevenue by Category:\n", top_categories)
else:
    print("\nNo 'Category' column found. Skipping category analysis.")
```



```
Top 5 Products by Revenue:
 Product
Chair      195620.11
Printer    195612.61
Laptop     192126.56
Tablet     186568.95
Monitor    175651.41
Name: TotalPrice, dtype: float64
```




```
No 'Category' column found. Skipping category analysis.
```

Project 2: E-Commerce EDA - Final Report

Dataset Overview: 1,200 orders, 14 columns, Jan 2023 - Jun 2025. The only missing values are in CouponCode (309 nulls, expected for orders without a coupon).

Methods Used:

1. IQR method for outlier detection
2. Z-Score method for comparison
3. Boxplot + Histogram for distribution analysis
4. Correlation heatmap for feature relationships

3 Key Insights:

1. **A high-value segment exists, and only IQR detected it:** IQR flagged 8 orders above 3,330.41 (Printer 2, Laptop 2, Tablet 2, Chair 1, Monitor 1). All 8 have the maximum quantity (5) combined with high unit prices (667-691). Z-Score flagged 0 outliers (max z = 2.93) because the right-skewed distribution inflates the standard deviation.
2. **Revenue is driven by both price and quantity:** TotalPrice correlates 0.72 with UnitPrice and 0.62 with Quantity, while Quantity and UnitPrice are almost uncorrelated (0.015). Each factor contributes to revenue independently.
3. **Top revenue products match the outliers:** Chair, Printer, Laptop, Tablet and Monitor generate the most revenue, and all 8 outlier orders come from exactly these 5 products.

Business Recommendation:

1. Create a dedicated high-value ("Enterprise") segment for these orders and assign an account manager.
2. Introduce bulk-purchase incentives on the top revenue products to encourage more high-quantity orders.
3. No pricing error found: the outliers combine valid quantities and unit prices within the observed ranges, so they are real orders and no data correction is needed.

In [ ]: